FASE 1 — LEVANTAMENTO DE REQUISITOS

Definições

Requisitos Funcionais

RF01. Detectar o fluxo de veículos em cada via
RF02. Ajustar automaticamente o tempo do sinal verde
RF03. Detectar presença de pedestres
RF04. Controlar sinal sonoro para pedestres
RF05. Identificar condições climáticas adversas
RF06. Priorizar veículos de emergência
RF07. Entrar em modo seguro em caso de falha
RF08. Enviar dados em tempo real para a central
RF09. Permitir configuração manual por operador autorizado

Requisitos Não Funcionais

RNF01. Tempo de resposta inferior a 2 segundos
RNF02. Disponibilidade mínima de 99%
RNF03. Sistema deve ser escalável para múltiplos cruzamentos
RNF04. Comunicação de dados criptografada
RNF05. Interface da central deve ser intuitiva
RNF06. Sistema deve operar sob temperaturas de -10°C a 50°C

Histórias de Usuário

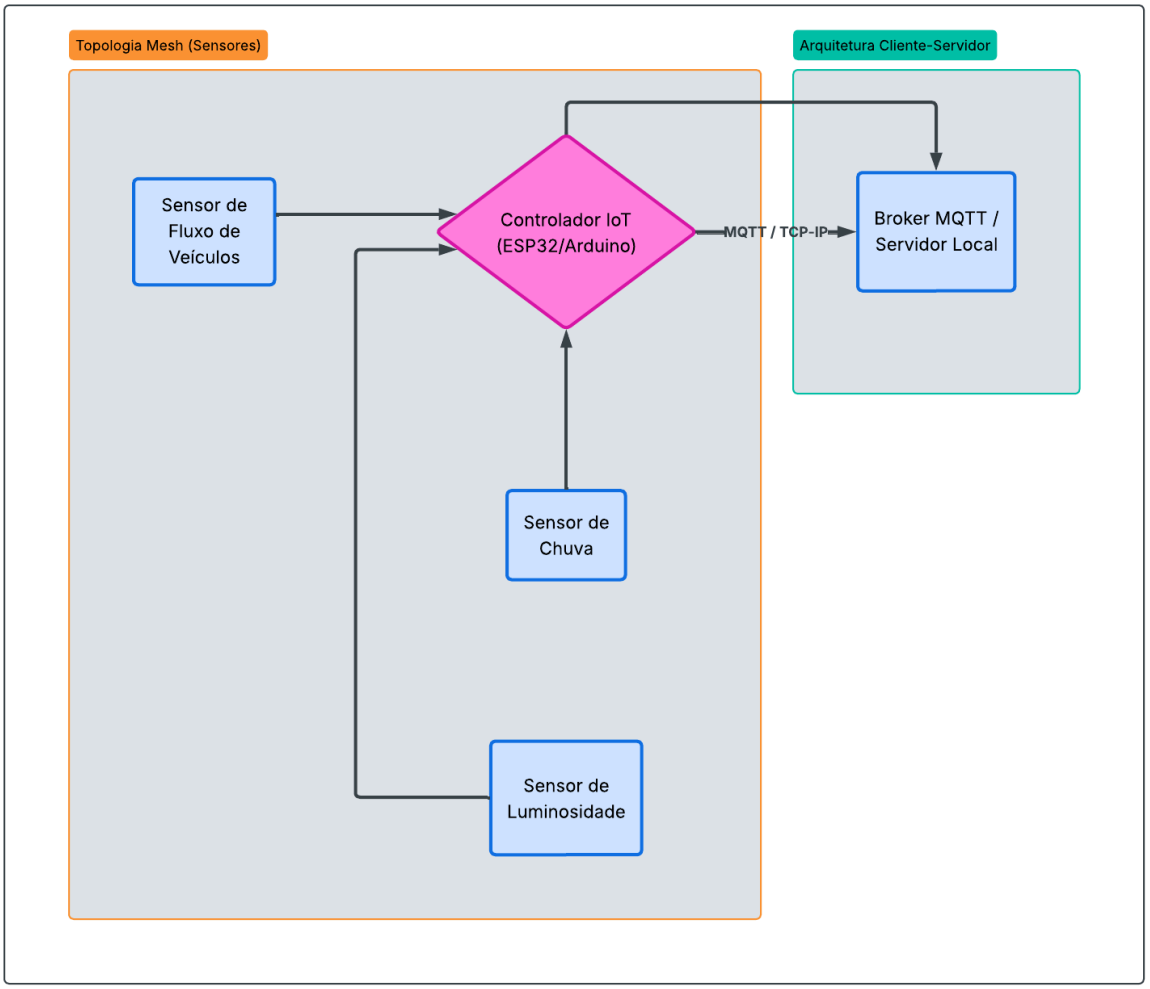
HU01
Como **motorista**,
quero que o semáforo ajuste o tempo de verde conforme o trânsito,
para reduzir congestionamentos.

HU02
Como **pedestre**,
quero tempo suficiente para atravessar com segurança,
mesmo em dias de chuva.

HU03
Como **operador da central**,
quero ser notificado quando um sensor falhar,
para agir rapidamente.

FASE 2 — MODELAGEM DO SISTEMA E ARQUITETURA IoT

- 1. Arquitetura de Rede IoT do cruzamento



Equipamentos de rede

1° Roteador - Interliga redes diferentes e encaminha pacotes entre elas
2° Switch - Conecta múltiplos dispositivos dentro da mesma rede local
3° Access Point - Fornece conectividade Wi-Fi aos dispositivos IoT.
4° Firewall - Controla e filtra o tráfego de dados entre redes.
5° Servidor Local - Processa, armazena e gerencia os dados da rede IoT.

FASE 3 — SISTEMA OPERACIONAL E SEGURANÇA

Escolher um SO para o servidor do semáforo

ITEM	WINDOWS SERVER	UBUNTU SERVER
CUSTO	ALTO	BAIXO
SEGURANÇA	MEDIO/ALTO	ALTO
SUPORTE A IOT	MEDIO	ALTO

Custo:

Windows: **Alto** - Licença paga por servidor, CALs por usuário e custos extras com suporte e ferramentas proprietárias. Pode chegar a valores elevados em ambientes corporativos.

Ubuntu: **Baixo** - Sistema operacional gratuito; custos adicionais podem surgir apenas se optar por suporte comercial (ex.: Ubuntu Advantage).

Segurança:

Windows: **Medio/Alto -** Ferramentas integradas (Defender, BitLocker, Firewall avançado), mas historicamente mais vulnerabilidades relatadas e alvo frequente.

Ubuntu: **Alto -** Arquitetura robusta, menos vulnerabilidades críticas relatadas e forte comunidade/empresas auxiliando no hardening (SELinux, AppArmor).

Suporte a IOT:

Windows: **Medio -** Suporte via Windows IoT está disponível e integrado ao ecossistema Microsoft, mas menos usado em servidores tradicionais.

Ubuntu: **Alto -** Ubuntu é amplamente usado em ambientes IoT.

2. Configurar em laboratório/VM:

CRIAR USUARIOS: Abrir cmd > digitar **net user “nome do usuario “senha” /add** > verificar usuario criado **net user**.

PERMISSÕES:Abrir painel de controle>conta de usuario>gerencia outra conta>e alterar o tipo de conta

PASTA COMPARTILHADA: crie uma nova pasta no local> Dê um nome à pasta>Clique com o botão direito na pasta>Propriedades>va ate a aba

compartilhamento> clique compartilhamento avançado> marque a caixa compartilhar a pasta>depois va na aba permissoes>escolher os usuarios que pode ler, modificar ou controle total

AJUSTAR VARIAVEIS DE AMBIENTE:Pesquisar e abrir Propriedades do sistema> avançado> variaveis de ambiente> dois cliques em path> editar>clicar na linha em branco> digitar C:\php.

TESTAR NAVEGAÇÃO VIA TERMINAL: Abrir cmd> digitar **ping.**

CONFIGURAR FIREWALL: Digite "Segurança do Window>Clique em "Firewall e proteção de rede"> Selecione o perfil para ajustar as regras.>Em "Microsoft Defender Firewall", deixe ativado>Clique em "Permitir um aplicativo ou recurso pelo Firewall do Windows" para permitir o acesso em redes específicas.

3. Criar uma mini Política de Segurança (PSI)

Senhas: Devem ter **mínimo de 8 caracteres**, letras maiúsculas e minúsculas, números, pelo menos 1 caractere especial e troca obrigatória a cada **90 dias**.

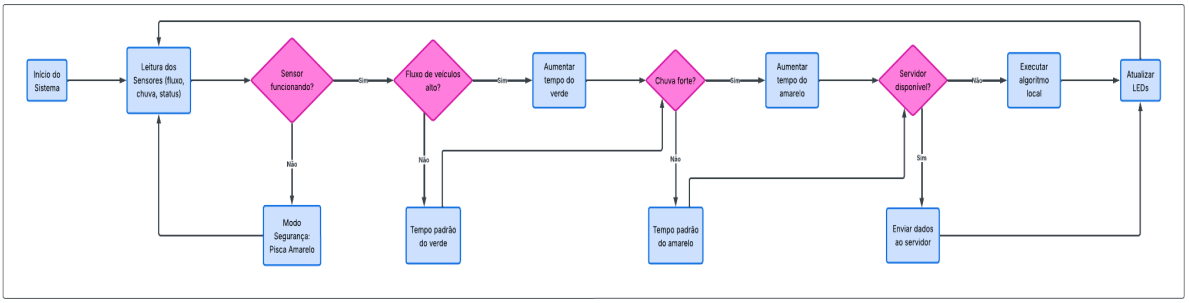
Acessos: Cada usuário deve possuir **login individual,** usuários administrativos (sudo/root) apenas quando necessário, contas de usuários inativos devem ser desativadas.

Backup: Devem ser realizados diariamente**,** dados críticos devem possuir backup automático**,** um backup local e um backup externo (nuvem) e acesso aos backups restrito a usuários autorizados.

Procedimentos em caso de falha no sensor: O sistema deve entrar automaticamente em modo seguro, utilizar dados históricos para manter operação básica, gerar alerta para a central de monitoramento
Registrar a falha em log do sistema
Técnico responsável deve avaliar e substituir o sensor, após correção, realizar teste antes de voltar à operação normal

Proteção contra engenharia social: Usuários devem ser treinados para não clicar em links suspeitos e nem fornecer senhas por e-mail, telefone ou mensagens

FASE 4 — ALGORITMO DO SEMÁFORO INTELIGENTE



Parte do código para a criação da parte visual

```python
# ========================
# CANVAS
# ========================
canvas = tk.Canvas(janela, width=420, height=380, bg="black")
canvas.pack(side="left", padx=10)

# ========================
# SEMÁFORO
# ========================
vermelho = canvas.create_oval(160, 60, 260, 160, fill="gray")
amarelo = canvas.create_oval(160, 170, 260, 270, fill="gray")
verde = canvas.create_oval(160, 280, 260, 380, fill="gray")

# ========================
# STATUS + TIMER
# ========================
status_texto = canvas.create_text(
    210, 25,
    text="Sistema iniciado",
    fill="white",
    font=("Arial", 12, "bold"),
    justify="center"
)

timer_texto = canvas.create_text(
    210, 50,
    text="⏱ Tempo: --",
    fill="cyan",
    font=("Arial", 11, "bold")
)

# ========================
# HISTÓRICO
# ========================
log_texto = tk.Text(
    janela,
    width=35,
    height=20,
    bg="black",
    fg="white",
    font=("Courier", 9)
)
log_texto.pack(side="right", padx=10)
log_texto.insert("end", "HISTÓRICO DO SISTEMA\n--------------------\n")
log_texto.config(state="disabled")
```

Parte que cria as funções do funcionamento do semáforo

```python
def decidir_verde():
    fluxo = sensor_fluxo()
    chuva = sensor_chuva()

    if fluxo > 30:
        verde_fluxo_alto()
    elif chuva:
        verde_chuva()
    else:
        verde_normal()

def verde_fluxo_alto():
    apagar_luzes()
    canvas.itemconfig(verde, fill="green")

    atualizar_status(
        "VERDE",
        "Fluxo alto de veículos detectado",
        "fluxo"
    )

    iniciar_timer(TEMPO_VERDE_ESTENDIDO)
    janela.after(TEMPO_VERDE_ESTENDIDO, semaforo_amarelo)

def verde_chuva():
    apagar_luzes()
    canvas.itemconfig(verde, fill="green")

    atualizar_status(
        "VERDE",
        "Chuva detectada – tráfego mais lento",
        "chuva"
    )

    iniciar_timer(TEMPO_VERDE_ESTENDIDO)
    janela.after(TEMPO_VERDE_ESTENDIDO, semaforo_amarelo)

def verde_normal():
    apagar_luzes()
    canvas.itemconfig(verde, fill="green")

    atualizar_status(
        "VERDE",
        "Fluxo normal de veículos",
        "normal"
    )

    iniciar_timer(TEMPO_VERDE)
    janela.after(TEMPO_VERDE, semaforo_amarelo)
```

```python
def semaforo_amarelo():
    apagar_luzes()
    canvas.itemconfig(amarelo, fill="yellow")

    atualizar_status(
        "AMARELO",
        "Transição de sinal",
        "normal"
    )

    iniciar_timer(TEMPO_AMARELO)
    janela.after(TEMPO_AMARELO, semaforo_vermelho)

def semaforo_vermelho():
    apagar_luzes()
    canvas.itemconfig(vermelho, fill="red")

    atualizar_status(
        "VERMELHO",
        "Parada obrigatória",
        "normal"
    )

    iniciar_timer(TEMPO_VERMELHO)
    janela.after(TEMPO_VERMELHO, decidir_verde)
```

Código feito em Py com a biblioteca Tkinter, que é um Framework do próprio Py que tem como função criar algo visual para o código funcionar

Todo o código foi feito usando “dados” aleatórios para simular um semáforo inteligente que de acordo com o fluxo de carros e o estado do clima, altera o tempo do verde, amarelo e vermelho

Para executar o código usar os comandos:

1. cd C:\Users\24004093821\Desktop\Back-End\Revisão> (Caminho da pasta do código)

2. python semaforo_visual.py

Link do Git: https://github.com/Scherrer01/Back-End.git
Biblioteca usada: Tkinter